

**Plan de Pruebas de Software Avanzado (Test Plan): Especificación Técnica y
Escenarios de Validación**

Jesus David Valencia León, Luis Miguel Bonett, y y Sebastian Albis

Ingeniería de Sistemas

Universidad de Pamplona

Ingeniería del Software II

Fanny Chiquillo

27 de mayo de 2026

Plan de Pruebas de Software Avanzado (Test Plan): Especificación Técnica y Escenarios de Validación

Introducción al Enfoque de Pruebas

El presente Plan de Pruebas de Software establece una estrategia de validación de caja blanca y caja negra para el "Sistema Web para la Gestión Automatizada de Turnos y Cálculo de Horas Extras". Dada la naturaleza transaccional y financiera del sistema, regido por la Ley 2101 de la República de Colombia, se exige un nivel de rigor técnico absoluto. Las pruebas no solo validarán el comportamiento de la Interfaz de Usuario (UI), sino la exactitud matemática del motor de cálculo (Backend) y la protección de los Endpoints (API REST).

Estrategia Tecnológica y Herramientas (Uso Real)

El aseguramiento de la calidad (QA) será liderado por el rol de Tester utilizando un *stack* de herramientas especializadas, aplicadas mediante las siguientes técnicas:

1. Postman (Pruebas de Integración y API REST)

Se utilizará Postman para validar los contratos de la API desarrollada en Flask. Se crearán *Collections* con variables de entorno (`{{base_url}}`) para aislar los entornos de Desarrollo y Producción.

- **Uso Específico:** Validación de códigos de estado HTTP. Por ejemplo, al inyectar un *payload* JSON para marcar entrada (`POST /api/v1/attendance/clock-in`), el script de prueba de Postman (escrito en JavaScript) validará automáticamente que la respuesta sea un código 201 `Created` y no un 200 `OK` genérico, asegurando además que el tiempo de respuesta (*Latency*) sea inferior a 500ms.

2. PyTest y PyTest-Cov (Pruebas Unitarias del Motor Lógico)

El motor matemático de la nómina será sometido a un *Test-Driven Development* (TDD) simulado.

- **Uso Específico:** Se empleará el decorador `@pytest.mark.parametrize` para inyectar matrices de fechas utilizando la técnica de "Análisis de Valores Límite" (*Boundary Value Analysis*). Esto es crítico para probar el límite exacto de

las 21:00 horas (9:00 PM), inyectando horas de salida como 20:59:59 y 21:00:01 para garantizar que el recargo nocturno del 35 % se dispare en el milisegundo correcto. Se exigirá un *Coverage* (Cobertura de código) mínimo del 90 % medido con la librería `pytest-cov`.

3. DBeaver y PostgreSQL (Auditoría de Integridad)

- **Uso Específico:** Durante las pruebas de transaccionalidad, se forzarán interrupciones en la mitad del proceso de guardado de un turno. Se utilizarán consultas nativas SQL en DBeaver para comprobar que los comandos `ROLLBACK` de SQLAlchemy en Flask hayan funcionado, garantizando que no queden registros huérfanos" (empleados con entrada pero sin registro de turno asociado en la base de datos).

Especificación Profunda de Casos de Prueba Críticos

A continuación, se documentan los escenarios de validación con un nivel de especificidad técnica orientado a la automatización.

Módulo de Seguridad y Autenticación (API)

Módulo de Asistencia (Manejo de Estados)

Módulo Core: Motor de Cálculo (Cruce de Fronteras Legales)

El siguiente caso evalúa la lógica del sistema frente a un escenario real y complejo de la legislación laboral.

Gestión de Defectos y Entregables de Calidad

Al finalizar las baterías de pruebas, se generarán dos entregables técnicos que se adjuntarán a la documentación final del proyecto:

1. **Reporte de Cobertura HTML (*Coverage Report*):** Un documento generado por PyTest que certificará qué líneas del código fuente en Python fueron ejecutadas por las pruebas y cuáles fueron ignoradas.
2. **Colección de Postman Exportada:** Un archivo `.json` alojado en el repositorio de GitHub (`/docs/postman_collection.json`) que permitirá a cualquier

evaluador o desarrollador futuro ejecutar todas las pruebas de la API con un solo clic.

ID Caso	Precondiciones	Pasos / Petición HTTP	Resultado Esperado Exacto
SEC-01 <i>Falta de Token JWT</i>	1. Base de datos activa. 2. API corriendo en localhost:5000. 3. Cliente sin sesión iniciada.	Endpoint: GET /api/v1/payroll/export Headers: Ninguno. Body: N/A	HTTP Status: 401 Unauthorized Response JSON: <pre>{.error": "Token missing", code": 401}</pre>

ID Caso	Precondiciones	Pasos / Petición HTTP	Resultado Esperado Exacto
ATT-01 <i>Salida sin Entrada</i>	1. Empleado ID=45 existe. 2. Empleado NO tiene registro de entrada abierto en la tabla <code>marcaciones</code> .	Endpoint: POST <code>/api/v1/attendance/clock-out</code> Body JSON: <code>{.emp_id": 45}</code> <i>(Nota: La hora se toma del backend, no del body).</i>	HTTP Status: 409 Conflict Response JSON: <code>{.error": "No open shift found to clock out", code": 409}</code>

ID Caso	Precondiciones	Datos de Ejecución (PyTest)	Resultado Esperado Exacto
CAL-01 <i>Extra</i> <i>Nocturna</i> <i>Limit</i>	1. Empleado con turno oficial asignado de 14:00 (2:00 PM) a 22:00 (10:00 PM) . 2. Salario base definido paramétricamente.	1. Inyectar <code>clock_in</code> = 2026-05-20 14:00:00 2. Inyectar <code>clock_out</code> = 2026-05-20 23:00:00 3. Ejecutar función <code>liquidar_turno()</code> .	Salida del Diccionario Python: - <code>ord_diurnas</code> : 7.0 <i>(14:00 a 21:00)</i> - <code>ord_nocturnas</code> : 1.0 <i>(21:00 a 22:00)</i> - <code>ext_nocturnas</code> : 1.0 <i>(22:00 a 23:00)</i>
CAL-02 <i>Tolerancia</i> <i>Marcaje</i>	1. Turno: 06:00 a 14:00 . 2. Configuración de gracia: 15 minutos en el archivo <code>.env</code> .	1. Inyectar <code>clock_in</code> = 05:48:00 2. Inyectar <code>clock_out</code> = 14:10:00 3. Ejecutar liquidación.	Salida del Diccionario Python: - <code>ord_diurnas</code> : 8.0 - <code>ext_diurnas</code> : 0.0 <i>(El sistema ignora minutos extra dentro del umbral de tolerancia).</i>